

**A REPORT
ON
STM32-BASED REAL-TIME DATA ACQUISITION AND LOGGING SYSTEM**

BY

Name(s) of the students
Atharva Abhiram Vemulapalli

ID No(s).
2024AAPS0211H

Discipline(s)
B. E in Electronics and
Communication
Engineering

**Prepared in partial fulfillment of the
Practice School - I**

AT

**Centre for Development of Advanced Computing (C-DAC), Hyderabad
A Practice School - I
Station of**



BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

July, 2026

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI
Practice School Division

Station
Centre for Development of
Advanced Computing (C-
DAC)

Centre
Hyderabad

Duration
25th May, 2026 - 18th July,
2026

Date of start
25th May, 2026

Date of submission
16th July, 2026

Title of the project: STM32-Based Real-Time Data Acquisition
and Logging System

**ID No.(s), Name(s) and Dis-
ciplines of the student(s):** 2024AAPS0211H
Atharva Abhiram Vemulapalli
B. E in Electronics and Communication Engineering

**Name(s) and Designation(s)
of the expert(s):** Dr. Prakash
Scientist

**Name(s) of the PS faculty
member(s):** Prof. Arindam Kushagra

Key words: STM32, Real-Time Monitoring, Data Acquisition,
Sensor Interfacing, UART, ADC, PWM, Data Logging

Project area(s): Embedded Systems and IoT

Abstract: This project presents the design and implementation
of an STM32-Based Real-Time Data Acquisition and
Logging System. The system periodically acquires
temperature, humidity, and ambient light data using
DHT11 and LDR sensors, processes the readings using
an STM32 microcontroller, and transmits them to a
host computer via UART for real-time monitoring and
CSV-based data logging.

Signature of the student(s)

Signature of the PS faculty member(s)

Date
16th July, 2026

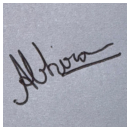


Date

Acknowledgements

I would like to express my sincere gratitude to Dr. Prakash, Scientist at C-DAC Hyderabad, for his invaluable guidance, encouragement, and continuous support throughout this project. His expertise in embedded systems and real-time data acquisition greatly contributed to the successful completion of this work.

I would also like to thank Prof. Arindam Kushagra and the Practice School Division of BITS Pilani for providing me with the opportunity to undertake this internship. The knowledge and practical experience gained during this project significantly enhanced my understanding of STM32 microcontrollers, embedded systems, sensor interfacing, real-time data acquisition and embedded firmware development.

A small, square, grayscale image of a handwritten signature in black ink on a light background. The signature is stylized and appears to be 'Arindam'.

Signature of the student(s)

Contents

1	Introduction	11
1.1	Background and Motivation	11
1.2	Project Objectives	11
1.3	Project Scope	12
1.4	Organization of the Report	12
2	Literature Review	13
2.1	Embedded Systems	13
2.2	STM32 Microcontrollers	13
2.3	Data Acquisition Systems	14
2.4	Sensor Interfacing	14
2.5	Analog-to-Digital Conversion	15
2.6	Timer-Based Sampling	15
2.7	Communication Interfaces	16
2.8	Data Logging Systems	16
2.9	Pulse Width Modulation (PWM)	17
3	Methodology	19
3.1	System Overview	19
3.2	System Architecture	19
3.3	Hardware Components	20
3.4	Software Tools	21
3.5	Development Methodology	21
3.5.1	Peripheral Configuration	22
3.5.2	Timer Configuration	22
3.5.3	Data Acquisition	23
3.5.4	Data Processing	23
3.5.5	Data Transmission	24
3.5.6	Data Logging	24
4	Results and Discussion	25
4.1	Experimental Setup	25
4.2	UART Communication Results	25
4.3	CSV Data Logging Results	26
4.4	Sensor Measurement Results	27
4.5	PWM-Based RGB LED Operation	27

4.6	Discussion	29
5	Conclusion and Future Work	31
5.1	Conclusion	31
5.2	Future Work	31
6	Recommendations	33
	Bibliography	35
	Appendices	37
A	Project Repository	39

List of Figures

2.1	Basic Architecture of an Embedded System	13
2.2	Generalized Data Acquisition System Architecture	14
2.3	Analog-to-Digital Conversion Process	15
2.4	Periodic Sensor Sampling Using Timer Interrupts	15
2.5	Proposed Data Logging Workflow	16
2.6	Pulse Width Modulation (PWM) Signal with Different Duty Cycles	17
2.7	Block Diagram of STM32 Microcontroller Architecture	18
3.1	Block Diagram of the Implemented STM32-Based Real-Time Data Acquisition and Logging System	19
3.2	Experimental Hardware Setup of the STM32-Based Data Acquisition System	20
3.3	Firmware Development Workflow Using STM32CubeMX and STM32CubeIDE	21
3.4	Flowchart of the Implemented Real-Time Data Acquisition and Logging Process	21
3.5	STM32CubeMX Pin Configuration for the Implemented Data Acquisition System	22
3.6	STM32CubeMX Configuration of TIM2 and ADC1 Peripherals	23
4.1	UART Terminal Display Showing Real-Time Sensor Data	25
4.2	CSV File Containing Logged Sensor Readings Generated by the STM32-Based Data Acquisition System	26
4.3	Ambient Light Intensity Recorded Over Time	27
4.4	RGB LED Color Indications in Light Mode Under Different Ambient Light Conditions	28
4.5	RGB LED Indication in Temperature Mode	28

List of Tables

2.1	Comparison of Sensors Used in Embedded Data Acquisition Systems	14
2.2	Comparison of UART, SPI, and I2C Communication Protocols	16
3.1	Hardware Components Used in the Implemented System	20
3.2	Software Tools Used During Development	21

Chapter 1

Introduction

Embedded systems play an increasingly important role in our world today, enabling us to observe, command and manipulate the data that surrounds us. At their core are data acquisition systems, which collect information from sensors, process it, and output it in a form suitable for monitoring, logging or further processing. They are used in a wide range of applications, from industrial control and automation to environmental monitoring, healthcare, smart homes and the Internet of Things (IoT).

Modern microcontrollers simplify the implementation of real-time data acquisition systems with built-in peripherals. ADCs, timers, communication interfaces and sophisticated interrupt controllers enable sensor interfacing, periodic sampling, data processing and reliable communication. The STM32 family stands out with its impressive set of peripherals and powerful processing subsystems, making it a top choice for embedded monitoring and data logging applications.

Our project focuses on designing and implementing an STM32-Based Real-Time Data Acquisition and Logging System (DALs). A detailed realization of the system is based on the STM32F446RE microcontroller. The implemented system periodically acquires temperature, humidity, and ambient light data using the DHT11 sensor and LDR. The acquired data are processed by the STM32 microcontroller and transmitted to a host computer through UART in CSV format for real-time monitoring and data logging.

1.1 Background and Motivation

The growing demand for automated systems has driven innovation in data acquisition technologies. Industries and research facilities alike seek reliable, real-time methods for data acquisition and logging. The availability of inexpensive and versatile microcontrollers presents an opportunity for them to act as core components in the construction of such systems.

As environmental sensing and monitoring needs escalate, developers focus on miniaturized, cost-effective embedded data acquisition systems.

This project implements such a system by interfacing with a sensor, periodically sampling and logging data via UART, all controlled by an STM32 microcontroller.

1.2 Project Objectives

The primary objectives of this project are:

- To design and implement a real-time data acquisition system using the STM32F446RE microcontroller.
- To interface DHT11 and LDR sensors for environmental monitoring.
- To implement timer-based periodic sampling and ADC-based sensor acquisition.
- To transmit sensor data to a host computer using UART communication.
- To store acquired data in CSV format for monitoring and analysis.
- To provide real-time visual indication of ambient light intensity using PWM-controlled RGB LEDs.

1.3 Project Scope

The scope of this project is to build a real-time data acquisition system using an STM32 microcontroller. This system interfaces with multiple sensors, collecting readings at regular intervals and transmitting data to a host computer. The developed system periodically acquires temperature, humidity, and ambient light data, processes the readings, and transmits them to a host computer for real-time monitoring and CSV-based logging. The modular design allows future extensions such as additional I2C/SPI sensors, SD card storage, wireless communication, cloud connectivity, and real-time visualization dashboards.

1.4 Organization of the Report

The remainder of this report is organized as follows. Chapter 2 presents a review of embedded systems, STM32 microcontrollers, and data acquisition techniques. Chapter 3 describes system design, hardware components, software tools, and implementation methodology. Chapter 4 presents the implementation results, experimental observations, and system performance. Chapter 5 concludes the work and discusses possible future enhancements.

Chapter 2

Literature Review

2.1 Embedded Systems

An embedded system is a system designed for a specific task or set of tasks, within larger electronic and mechanical system [2]. Unlike desktop computers, it is a single-purpose device, often required to perform under real-time constraints. In short, they are everywhere. From consumer electronics to industrial control, in vehicles, medical devices, aeronautics and the Internet of Things.

An embedded system typically consists of one or more microprocessors or microcontrollers, memory, input/output connections, sensors, actuators and software. The hardware and software work together to monitor and control the system, communicate with other devices and process relevant data from the surrounding environment.

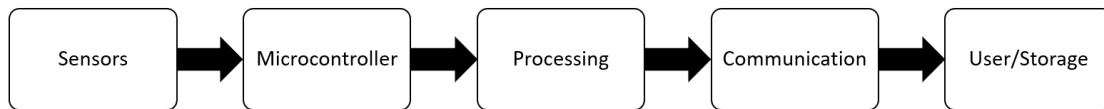


Figure 2.1: Basic Architecture of an Embedded System

Key goals are low power consumption, reliability, compactness and real-time capability, features that make them suitable for data logging and monitoring.

2.2 STM32 Microcontrollers

STM32 is a family of 32-bit microcontrollers developed by STMicroelectronics and based on ARM Cortex-M processor cores [4]. They have gained widespread acceptance in the embedded domain for their powerful capabilities, energy efficiency, and a variety of peripheral supports.

The STM32 devices feature several on-chip peripheral components such as General-Purpose Input Output (GPIO) ports, Analog-to-Digital Converters (ADC), timers, Universal Asynchronous Receiver Transmitter (UART), Serial Peripheral Interface (SPI), Inter-Integrated Circuit (I2C), Direct Memory Access (DMA) and interrupt controllers [5]. The STM32F446 microcontroller architecture shown in Figure 2.7 is adapted from the STM32F446 datasheet published by STMicroelectronics [4].

The ARM Cortex-M architecture is designed for efficient instruction processing, effective interruption handling and effective power management. Tools like STM32CubeMX and STM32CubeIDE are helpful for both hardware configuration and firmware development, thus STM32 microcontrollers would be suitable for education, research and industrial purposes.

2.3 Data Acquisition Systems

Data Acquisition Systems (DAQ) are primarily used for the collection, processing and storage of information acquired from physical sensors and transducers [3]. DAQ systems bridge the physical world and digital computers.

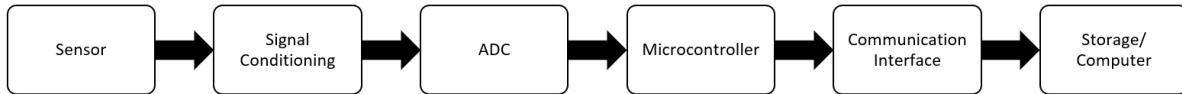


Figure 2.2: Generalized Data Acquisition System Architecture

The core elements of DAQ systems are sensors, signal conditioning circuits, analog-to-digital converters, a processing unit, a communication interface and a storage element. Data, after acquisition, can be used for display, analysis, logging or transmission to remote locations for monitoring. DAQ systems are used in industrial process control, environmental monitoring, health care instrumentation, lab experiments and smart structures. Their reliability and performance depend on the type of sensors chosen, a proper sampling technique and a reliable communication architecture.

2.4 Sensor Interfacing

Sensors are devices that convert physical parameters like temperature, humidity, light intensity, pressure and voltage into electrical signals. Proper interfacing is crucial in embedded system design.

Sensors fall into two categories: analog and digital [1]. Analog sensors output a voltage or current proportional to the input physical parameter. Analog signals must be converted to digital format before processing. In contrast, digital sensors provide processed information on interfaces like UART, SPI or I2C.

Sensor Type	Output	Interface
LDR	Analog	ADC
DHT11	Digital	Single Wire
BMP280	Digital	I2C/SPI

Table 2.1: Comparison of Sensors Used in Embedded Data Acquisition Systems

Signal conditioning, calibration, sampling and communication are critical when interfacing sensors. The correct use of sensor interfacing methods improves the accuracy of the measurement and the overall reliability of the system.

In this project, an LDR was interfaced through the STM32 ADC peripheral to measure ambient light intensity, while the DHT11 sensor was used to acquire temperature and humidity data. Together, these sensors demonstrate the integration of both analog and digital sensing techniques in a real-time embedded data acquisition system.

2.5 Analog-to-Digital Conversion

A number of physical parameters need to be read by sensors which are generating analog values, microcontrollers process digital data so the analog values need to be converted into binary values using an Analog-to-Digital converter (ADC).

The ADC takes readings of an analog signal at discrete points in time and then converts the signal into a binary value. The resolution of an ADC determines the accuracy of the digital signal that the ADC outputs, high resolution gives an accurate measurement.

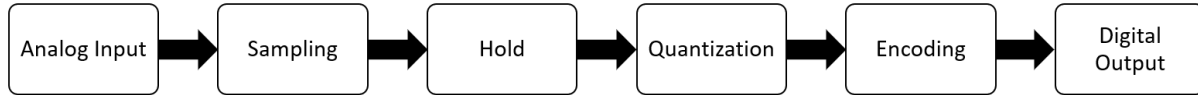


Figure 2.3: Analog-to-Digital Conversion Process

STM32 microcontrollers contain advanced ADCs capable of reading from various input channels with individually controllable sample time parameters [5], allowing for efficient reading from sensors during real-time operation. Analog-to-digital conversion has a direct impact on measurement accuracy, and the quality of data acquisition systems.

2.6 Timer-Based Sampling

Sampling involves reading a signal at regular intervals of time. Reliable data from sensors requires a reliable, stable sampling procedure.

Microcontroller timers are used to generate recurring events that initiate data acquisition. Timer-based sampling ensures smooth operation and eliminates the problem of sampling interval irregularities caused by software execution delays.

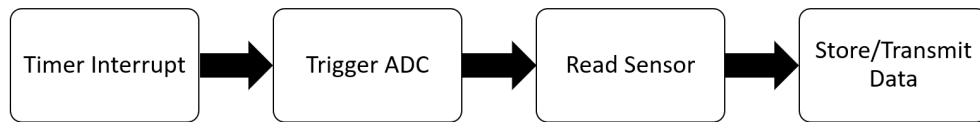


Figure 2.4: Periodic Sensor Sampling Using Timer Interrupts

In case of STM32 microcontrollers, a variety of timers are available, all with a selectable prescaler and an auto-reload register which can be used to provide exactly timed interrupt signals and thus can monitor the sensors and conduct data acquisition process in real time [5].

Using timers for sampling can improve system reliability and is a standard practice in various industrial control systems, measuring instruments and monitoring devices.

2.7 Communication Interfaces

Communication interfaces are crucial for microcontrollers, sensors, and various external devices to exchange data. In today's world of embedded systems, you will most likely come across communication protocols like UART, SPI and I2C [5].

UART (Universal Asynchronous Receiver-Transmitter) is a simple serial communication protocol which allows microcontrollers to communicate with computers. In this project, UART communication is used to transmit sensor readings from the STM32 microcontroller to a host computer, where the data is displayed and stored in CSV format for further analysis.

Another serial communication protocol is SPI (Serial Peripheral Interface). SPI is a synchronous serial communication interface which enables fast data communication between a master and multiple slave devices. It's often utilized when interfacing with sensors, memories and displays.

The last protocol of importance is I2C (Inter-Integrated Circuit). I2C is a two-wire interface protocol which enables several devices to use the same bus. Its simplicity and low pin count make it a good fit for interfacing with sensors and peripheral devices.

Feature	UART	SPI	I2C
Speed	Medium	High	Medium
Wires	2	4+	2
Complexity	Low	Medium	Medium
Typical Use	PC Communication	Sensor/Memory	Multiple Sensors

Table 2.2: Comparison of UART, SPI, and I2C Communication Protocols

2.8 Data Logging Systems

Data logging involves recording useful information, which can be used for analysis, display, or decision making [3]. They may be kept locally in memory devices or stored remotely in a different system (e.g., computer or cloud).

In a data logging setup, sensor readings are recorded with timestamps and other data. This information helps track trends, identify abnormalities, evaluate system performance, and predict maintenance needs.



Figure 2.5: Proposed Data Logging Workflow

In real-time monitoring applications, data logging enables long-term storage and analysis of acquired sensor data. In this project, sensor readings are transmitted over UART and stored in CSV format, allowing the collected information to be visualized and analyzed using spreadsheet software or plotting tools. Efficient logging mechanisms significantly improve the usefulness of embedded data acquisition systems.

2.9 Pulse Width Modulation (PWM)

Pulse Width Modulation (PWM) is a technique used to control the average power delivered to an electrical load by varying the duty cycle of a periodic digital signal while maintaining a constant frequency. By adjusting the ratio of the ON time to the total signal period, PWM enables efficient control of devices such as LEDs, motors, and power converters without significant energy loss.

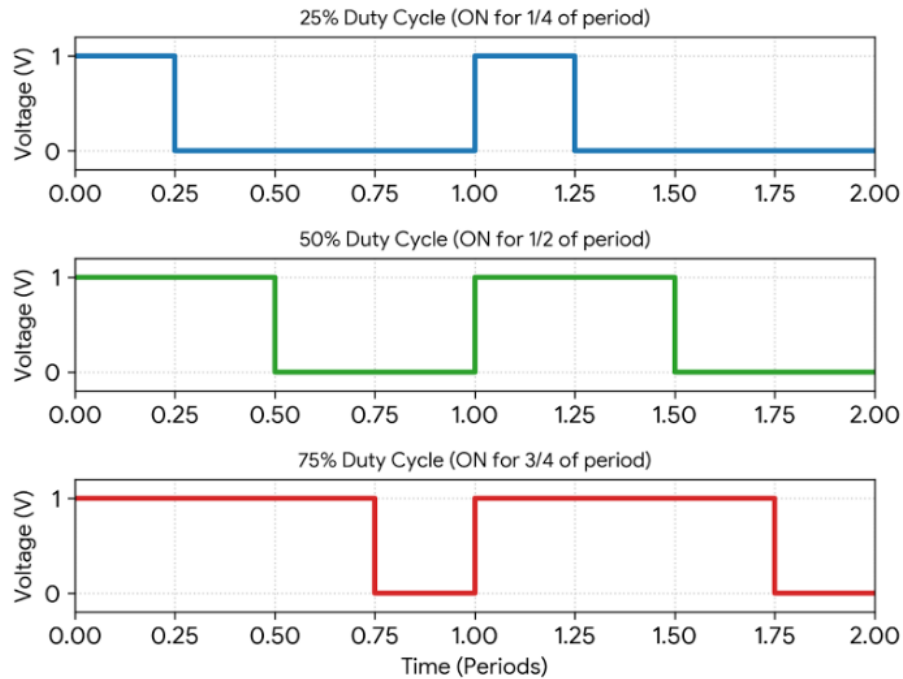


Figure 2.6: Pulse Width Modulation (PWM) Signal with Different Duty Cycles

STM32 microcontrollers provide dedicated timer peripherals capable of generating high-resolution PWM signals through multiple output channels. The duty cycle of the PWM signal can be dynamically modified in software, making it suitable for applications that require variable output control.

Figure 2.6 illustrates PWM signals with different duty cycles. Increasing the duty cycle increases the average voltage delivered to the load, enabling smooth control of devices such as LEDs and DC motors.

In this project, PWM is used to control the brightness and colour of an RGB LED based on the ambient light intensity measured by the LDR sensor. As the measured light intensity changes, the PWM duty cycle is adjusted accordingly, providing real-time visual feedback of the sensor readings. This demonstrates the practical use of STM32 timer peripherals for efficient output control in embedded systems.

Chapter 3

Methodology

3.1 System Overview

In this project, a Real-Time Data Acquisition and Logging System centers on an STM32F446RE microcontroller. It acquires environment details from a DHT11 sensor and a light-dependent resistor (LDR) by polling them over a periodic interval. The LDR's readings are processed by the STM32's ADC module while the DHT11 interacts through a one-wire digital protocol.

A timer is enabled to achieve data retrieval at a regular interval that triggers sensor reading. The microcontroller transmits sensor data via UART to the PC, where the data are stored in a CSV file for real-time monitoring and analysis. The RGB LED is connected for output, driven by PWM to reflect changing light conditions.

3.2 System Architecture

The overall architecture of the implemented system is shown in Figure 3.1.

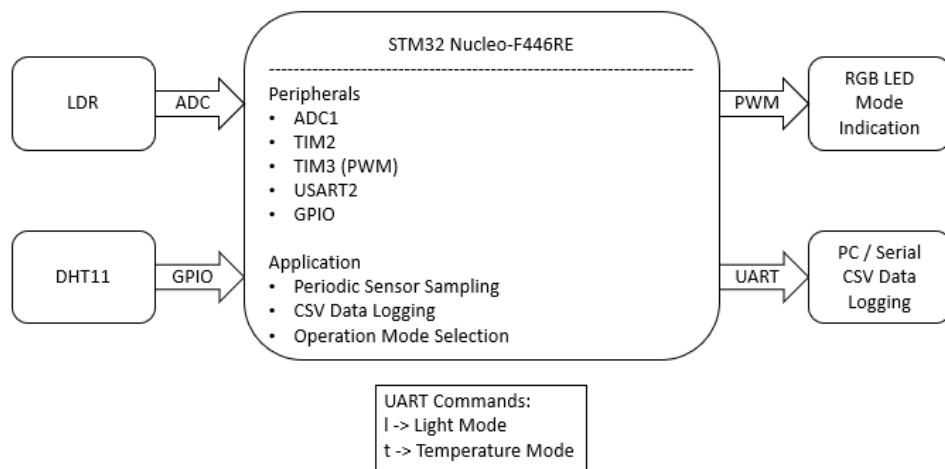


Figure 3.1: Block Diagram of the Implemented STM32-Based Real-Time Data Acquisition and Logging System

The DHT11 sensor provides temperature and humidity data through a single-wire digital

interface, while the LDR measures ambient light intensity using the STM32 ADC peripheral. The STM32F446RE periodically acquires sensor data using timer-based sampling, processes the readings, and transmits them to a host computer through UART communication. The received data is stored in CSV format for monitoring and analysis. Additionally, PWM-generated signals control an RGB LED to provide real-time visual indication of the measured light intensity.

3.3 Hardware Components

Component	Purpose
STM32 Nucleo-F446RE	Processing Unit
LDR	Light Intensity Measurement
DHT11	Temperature and Humidity Measurement
RGB LED	Visual Mode Indication
USB Cable	Programming and UART Communication
Breadboard	Circuit Prototyping
Jumper Wires	Hardware Connections

Table 3.1: Hardware Components Used in the Implemented System

The hardware platform consists of an STM32 Nucleo-F446RE development board interfaced with a DHT11 sensor, an LDR, and an RGB LED. The STM32F446RE serves as the central processing unit, handling sensor acquisition, data processing, PWM generation, and UART communication [4]. The DHT11 measures temperature and humidity, while the LDR monitors ambient light intensity through the ADC peripheral. An RGB LED provides visual indication of the selected operating mode through PWM control and the complete circuit is assembled on a breadboard using jumper wires.

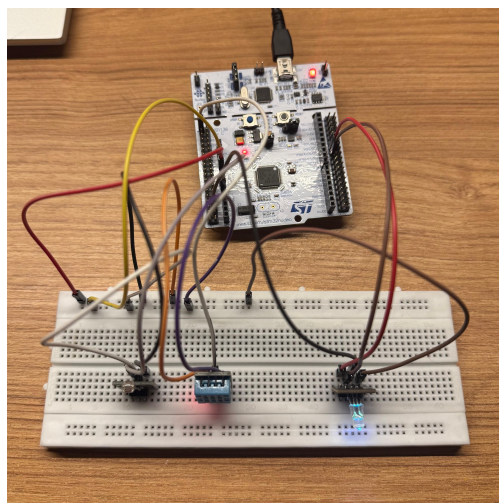


Figure 3.2: Experimental Hardware Setup of the STM32-Based Data Acquisition System

3.4 Software Tools

The software tools used during development are listed in Table 3.2.

Tool	Purpose
STM32CubeMX	Peripheral Configuration
STM32CubeIDE	Firmware Development, Compilation and Debugging
Embedded C	Firmware Development
Serial Terminal(PuTTY)	UART Data Monitoring
Microsoft Excel	CSV Data Logging and Visualization

Table 3.2: Software Tools Used During Development

STM32CubeMX was used to configure the STM32 peripherals, including GPIO, ADC, timers, UART, and PWM, and to generate the initialization code. STM32CubeIDE was used to develop, compile, debug, and program the firmware written in Embedded C. PuTTY was used to monitor UART communication, while Microsoft Excel was used to store the received data in CSV format and visualize the logged sensor readings.

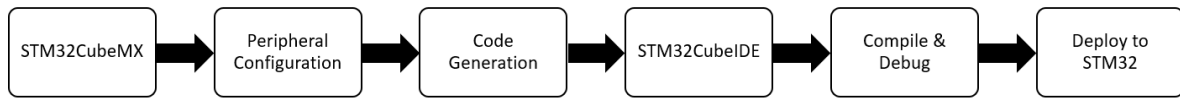


Figure 3.3: Firmware Development Workflow Using STM32CubeMX and STM32CubeIDE

3.5 Development Methodology

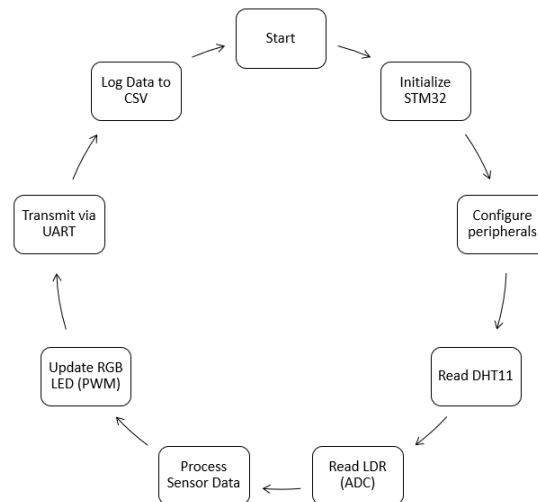


Figure 3.4: Flowchart of the Implemented Real-Time Data Acquisition and Logging Process

The firmware was developed in a modular manner, with each module responsible for a specific function of the data acquisition system. Figure 3.4 illustrates the overall execution flow, from peripheral initialization to sensor acquisition, data processing, UART communication, and CSV data logging.

3.5.1 Peripheral Configuration

STM32CubeMX was used to configure the peripherals required for the implementation of the data acquisition system. GPIO pins were configured to interface with the DHT11 sensor, RGB LED, and user push button. ADC1 was configured to acquire analog data from the LDR sensor. TIM3 was configured in PWM mode to control the RGB LED and USART2 was configured for UART communication with the host computer. After peripheral configuration, STM32CubeMX generated the initialization code, which served as the foundation for firmware development in STM32CubeIDE.

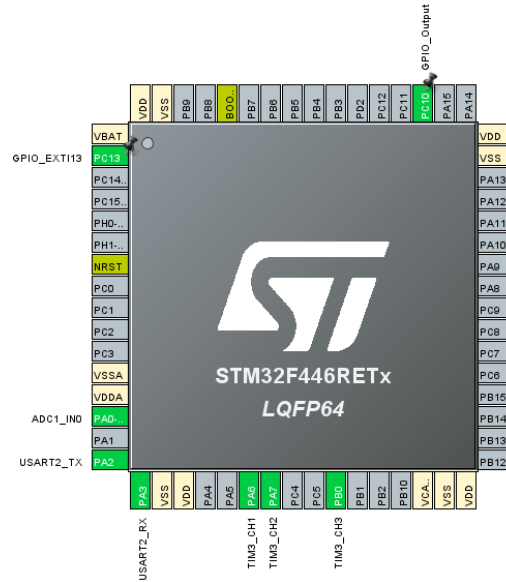


Figure 3.5: STM32CubeMX Pin Configuration for the Implemented Data Acquisition System

3.5.2 Timer Configuration

TIM2 was configured to generate periodic interrupts at fixed intervals, enabling deterministic sensor sampling. The timer prescaler and auto-reload register (ARR) values were selected to produce an interrupt every 1 second. Each timer interrupt invokes the interrupt service routine, where the sensor acquisition process is initiated.

Using timer-driven sampling ensures that sensor readings are acquired at regular intervals independent of the execution time of the main program. This approach improves measurement consistency and is widely adopted in real-time embedded systems for reliable data acquisition. Figure 3.6 shows the TIM2 configuration implemented using STM32CubeMX.

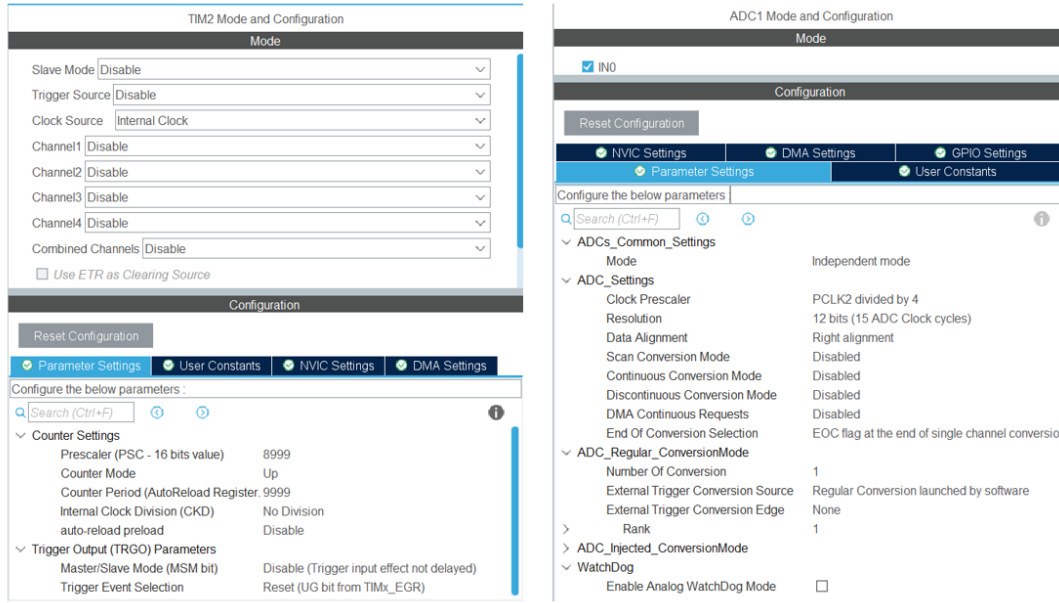


Figure 3.6: STM32CubeMX Configuration of TIM2 and ADC1 Peripherals

3.5.3 Data Acquisition

The implemented system acquires environmental data from two sensors: a DHT11 sensor and an LDR. The DHT11 provides digital temperature and humidity measurements through a single-wire communication interface, while the LDR produces an analog voltage proportional to the ambient light intensity.

The analog output of the LDR is sampled using the STM32 ADC1 peripheral, which converts the analog voltage into a 12-bit digital value for further processing. The ADC1 configuration used for analog data acquisition from the LDR sensor is also shown in Figure 3.6. Sensor acquisition is performed periodically within the timer interrupt routine, ensuring that all measurements are obtained at regular one-second intervals.

3.5.4 Data Processing

After acquisition, the sensor readings are processed by the STM32F446RE before transmission. The analog value obtained from the LDR is converted into a digital representation of the ambient light intensity, while the DHT11 provides temperature and humidity values in digital form. The acquired data is formatted into a comma-separated (CSV) structure suitable for serial transmission and storage on the host computer.

The processed light intensity value is also used to determine the PWM duty cycle of the RGB LED, providing real-time visual indication of the measured ambient light level. The firmware supports multiple operating modes, allowing either light intensity or temperature measurements to be monitored through UART commands.

3.5.5 Data Transmission

USART2 was configured to establish serial communication between the STM32F446RE and the host computer. After processing, the sensor readings were formatted into CSV-compatible strings and transmitted using the HAL UART library. The transmitted data included temperature, humidity, and light intensity measurements, enabling real-time monitoring on the host computer. The UART interface served as the communication link between the embedded system and the data logging application running on the PC [5].

3.5.6 Data Logging

The sensor data transmitted over UART was received by the host computer and stored in CSV (Comma-Separated Values) format. Each record contained the acquired temperature, humidity, and ambient light measurements, allowing the collected data to be easily viewed, processed, and analyzed using spreadsheet software.

CSV-based logging provides a lightweight and portable method for storing measurement data while maintaining compatibility with a wide range of analysis and visualization tools. The logged data can be used to observe environmental trends, evaluate system performance, and generate graphical representations of the acquired sensor readings.

Chapter 4

Results and Discussion

4.1 Experimental Setup

The STM32-based Real-Time Data Acquisition System was tested with the hardware described in Chapter 3. The STM32 Nucleo-F446RE recorded temperature, humidity, and ambient light readings from the DHT11 sensor and LDR at 1-second intervals. These data were subsequently processed by the microcontroller and transmitted via USART2 using UART communication protocol to a host computer.

Sensor data were monitored using PuTTY and logged into a CSV file for future analysis.

The system was validated by observing live readings, successfully logged CSV file and the PWM-driven RGB LED's operation in varied environments.

4.2 UART Communication Results

The implemented system was able to send sensor readings (temperature, humidity, and ambient light) from the STM32F446RE to the host computer via USART2 communication. The readings were transmitted in a CSV-compatible structured format. An application on the host computer was able to read the incoming stream of data in real-time using a serial terminal (PuTTY).

```
Light(l) or temperature(t)?
Time(ms),ADC Value,Duty Cycle,Temperature(C),Humidity(%),System Mode,LED Mode
0,451,1102,29,62,LIGHT,WHITE
1000,448,1094,29,62,LIGHT,WHITE
2000,448,1094,29,62,LIGHT,WHITE
3000,450,1099,29,62,LIGHT,WHITE
4000,460,1124,29,62,LIGHT,WHITE
5000,454,1109,29,62,LIGHT,WHITE
6000,454,1109,29,62,LIGHT,RED
7000,453,1107,29,62,LIGHT,RED
8000,456,1114,29,62,LIGHT,GREEN
9000,454,1109,29,62,LIGHT,BLUE
10000,453,1107,29,62,LIGHT,WHITE
11000,454,1109,29,62,LIGHT,WHITE
12000,458,1119,29,62,TEMP,-
13000,455,1111,29,62,TEMP,-
14000,455,1111,29,62,TEMP,-
15000,453,1107,29,62,TEMP,-
16000,454,1109,29,62,TEMP,-
17000,451,1102,29,62,TEMP,-
```

Figure 4.1: UART Terminal Display Showing Real-Time Sensor Data

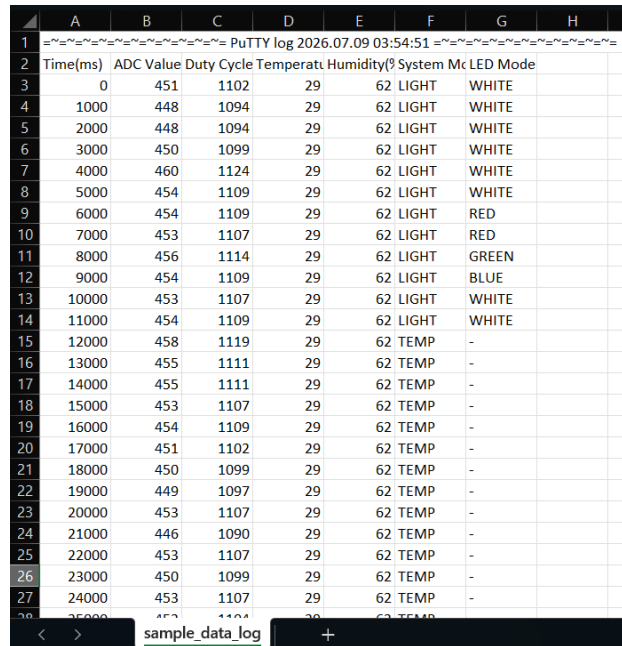
Successful communication between the embedded system and the host computer was verified.

The sensor values read by the serial terminal were updating at approximately 1-second intervals, which corresponds to the sample rate configured on TIM2. Continuous sampling of sensor data was working as expected, and the UART interface was verified. Figure 4.1 shows the real-time sensor readings from the host computer.

4.3 CSV Data Logging Results

The sensor data transmitted through UART was successfully stored in CSV (Comma-Separated Values) format on the host computer. Each record includes the sampling time, ADC value, PWM duty cycle, temperature, humidity, operating mode, and RGB LED status. The structured format enables efficient storage, analysis, and visualization of the acquired data.

The generated CSV file was verified using Microsoft Excel, where the logged values were organized into tabular form for further analysis. Successful CSV logging confirmed reliable data transfer from the STM32 microcontroller to the host computer while preserving the integrity of the acquired measurements.



Time(ms)	ADC Value	Duty Cycle	Temperature	Humidity	System Mode	RGB LED Mode
0	451	1102	29	62	LIGHT	WHITE
1000	448	1094	29	62	LIGHT	WHITE
2000	448	1094	29	62	LIGHT	WHITE
3000	450	1099	29	62	LIGHT	WHITE
4000	460	1124	29	62	LIGHT	WHITE
5000	454	1109	29	62	LIGHT	WHITE
6000	454	1109	29	62	LIGHT	RED
7000	453	1107	29	62	LIGHT	RED
8000	456	1114	29	62	LIGHT	GREEN
9000	454	1109	29	62	LIGHT	BLUE
10000	453	1107	29	62	LIGHT	WHITE
11000	454	1109	29	62	LIGHT	WHITE
12000	458	1119	29	62	TEMP	-
13000	455	1111	29	62	TEMP	-
14000	455	1111	29	62	TEMP	-
15000	453	1107	29	62	TEMP	-
16000	454	1109	29	62	TEMP	-
17000	451	1102	29	62	TEMP	-
18000	450	1099	29	62	TEMP	-
19000	449	1097	29	62	TEMP	-
20000	453	1107	29	62	TEMP	-
21000	446	1090	29	62	TEMP	-
22000	453	1107	29	62	TEMP	-
23000	450	1099	29	62	TEMP	-
24000	453	1107	29	62	TEMP	-

Figure 4.2: CSV File Containing Logged Sensor Readings Generated by the STM32-Based Data Acquisition System

Figure 4.2 shows a portion of the generated CSV file. The recorded data confirms successful logging of all measured and processed parameters, including sensor readings, operating mode, and RGB LED status. The consistent one-second timestamps demonstrate that the timer-based sampling mechanism operated as expected throughout the experiment.

4.4 Sensor Measurement Results

The logged sensor data was analyzed using Microsoft Excel to evaluate the performance of the implemented data acquisition system. Ambient light intensity measured by the LDR sensor was plotted against time to observe the variation in sensor readings during system operation. Temperature and humidity values obtained from the DHT11 sensor remained nearly constant throughout the experiment, indicating stable environmental conditions.

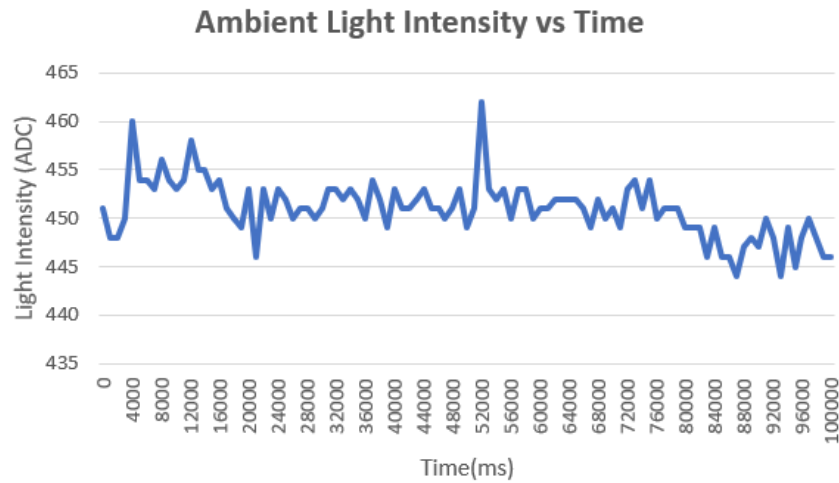


Figure 4.3: Ambient Light Intensity Recorded Over Time

Figure 4.3 shows the ambient light intensity recorded by the LDR sensor over time. The ADC values varied between approximately 444 and 462 counts, with only minor fluctuations during the experiment. These variations correspond to small changes in the surrounding light conditions and demonstrate the correct operation of the ADC peripheral and timer-based sampling mechanism. The consistent trend confirms that the implemented system reliably acquires, processes, and logs analog sensor data in real time.

The DHT11 sensor consistently measured approximately 29 °C and 62% relative humidity throughout the experiment, indicating stable environmental conditions and reliable digital sensor operation.

4.5 PWM-Based RGB LED Operation

The RGB LED was controlled using the PWM outputs of TIM3 to provide real-time visual feedback of the system status. In Light Mode, the LED changes its color according to the measured ambient light intensity, allowing users to identify different lighting conditions without referring to the serial output. Figure 4.4(a)-(d) show the RGB LED displaying different colors corresponding to different light intensity ranges.

The PWM-based RGB LED indication provides an intuitive visual representation of the system status without requiring continuous observation of the serial terminal. This feature enhances the usability of the data acquisition system by enabling users to quickly identify changes in ambient light conditions and the selected operating mode. The smooth color

transitions generated through PWM also demonstrate the flexibility of the STM32 timer peripherals for real-time embedded control applications.

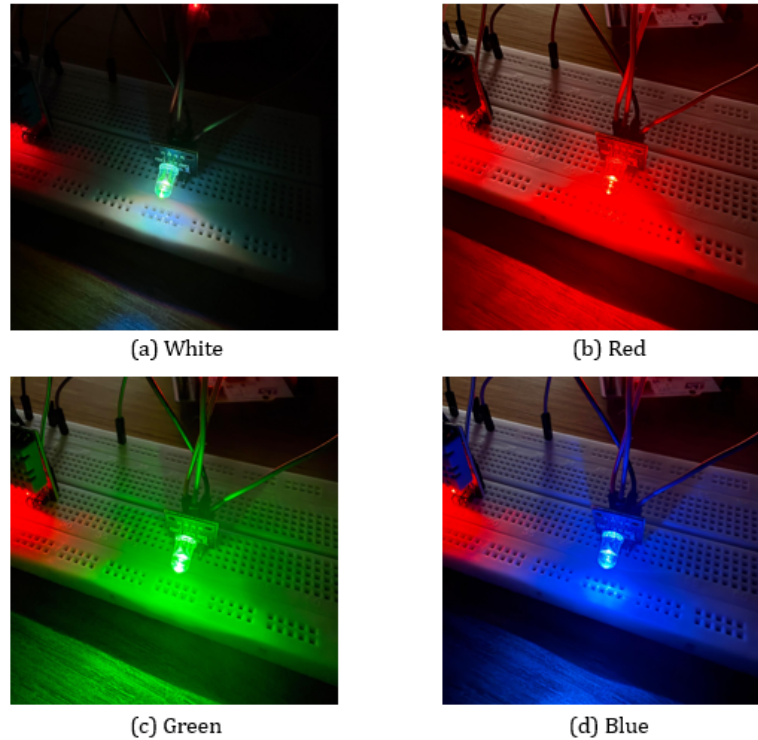


Figure 4.4: RGB LED Color Indications in Light Mode Under Different Ambient Light Conditions

Figure 4.5 shows the RGB LED operating in Temperature Mode. In this mode, the LED provides a visual indication corresponding to temperature monitoring rather than ambient light intensity. The successful mode transition demonstrates the correct implementation of firmware-based operating mode selection through UART commands.

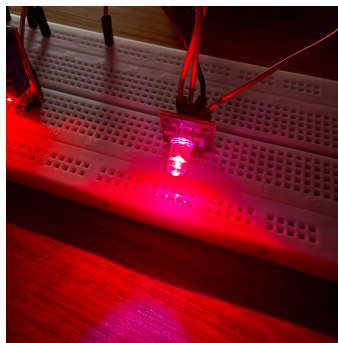


Figure 4.5: RGB LED Indication in Temperature Mode

4.6 Discussion

The experimental results demonstrate the successful implementation of the STM32-based Real-Time Data Acquisition and Logging System. Periodic sensor sampling using TIM2, analog-to-digital conversion through ADC1, and digital acquisition from the DHT11 sensor operated reliably throughout the experiments. The acquired data was successfully processed, transmitted to the host computer through UART, and stored in CSV format for subsequent analysis.

The generated sensor measurements and graphical analysis confirmed stable system operation under normal environmental conditions. The PWM-controlled RGB LED provided effective real-time visual feedback by indicating different ambient light levels and supporting a separate temperature monitoring mode. These results verify the correct integration of the STM32 peripherals and demonstrate that the implemented system satisfies the project objectives of real-time data acquisition, monitoring, visualization, and logging.

Chapter 5

Conclusion and Future Work

5.1 Conclusion

This project successfully designed and implemented an STM32-based Real-Time Data Acquisition and Logging System using the STM32 Nucleo-F446RE development board. The system periodically acquired environmental data from an LDR and a DHT11 sensor, processed the measurements using the STM32 microcontroller, and transmitted the acquired data to a host computer through UART communication. The received data was successfully stored in CSV format, enabling further analysis and visualization.

The implementation integrated several important STM32 peripherals, including GPIO, ADC, timers, UART, and PWM. Timer-based sampling ensured deterministic data acquisition, while PWM-controlled RGB LED indication provided real-time visual feedback of the measured parameters and operating modes. Experimental results confirmed reliable sensor interfacing, communication, and data logging throughout the project.

This project provided practical experience in embedded systems design, firmware development using STM32CubeMX and STM32CubeIDE, peripheral configuration, sensor interfacing, and real-time data acquisition techniques. The developed system achieved the objectives defined at the beginning of the internship and provides a solid foundation for more advanced embedded monitoring applications.

5.2 Future Work

The developed system can be extended further to improve its functionality and applicability. Possible future enhancements include:

- Interface additional sensors using I²C and SPI communication protocols.
- Replace UART communication with wireless interfaces such as Bluetooth or Wi-Fi for remote monitoring.
- Integrate an SD card module for standalone data logging without requiring a host computer.
- Develop a graphical user interface (GUI) for real-time visualization and analysis of sensor data.

- Implement cloud-based data storage and IoT connectivity for remote access and monitoring.
- Add alarm and notification mechanisms based on configurable sensor thresholds.
- Improve power efficiency by implementing low-power operating modes of the STM32 microcontroller.

Chapter 6

Recommendations

Based on the implementation and testing of the developed system, the following recommendations are suggested for future development and deployment:

- Calibrate sensors before deployment to improve measurement accuracy and reliability.
- Replace the DHT11 sensor with higher-accuracy sensors such as the DHT22 or BME280 for applications requiring precise environmental monitoring.
- Implement DMA-based ADC and UART communication to reduce CPU utilization and improve overall system performance.
- Incorporate robust error handling and fault detection mechanisms for improved system reliability.
- Develop a dedicated graphical user interface (GUI) to simplify real-time monitoring and data visualization.
- Use an SD card or external flash memory for standalone data logging in applications where a host computer is unavailable.
- Perform long-duration testing under different environmental conditions to evaluate the stability and robustness of the system.

Implementing these recommendations will further improve the scalability, reliability, and practical applicability of the developed STM32-based data acquisition system.

Bibliography

- [1] Jacob Fraden. *Handbook of Modern Sensors*. Springer, 5 edition, 2016.
- [2] Raj Kamal. *Embedded Systems: Architecture, Programming and Design*. McGraw Hill, 3 edition, 2017.
- [3] John Park and Steve Mackay. *Practical Data Acquisition for Instrumentation and Control Systems*. Newnes, 2003.
- [4] STMicroelectronics. *STM32F446xC/E Datasheet*. STMicroelectronics, 2023. Accessed: June 2026.
- [5] STMicroelectronics. *STM32F446xx Advanced ARM-based 32-bit MCUs Reference Manual (RM0390)*. STMicroelectronics, 2023. Accessed: June 2026.

Appendices

Appendix A

Project Repository

The complete source code, documentation, and demonstration images for this project are available at:

<https://github.com/atharvavemulapalli-lgtm/stm32-real-time-data-acquisition-logging-system>

The repository is maintained to facilitate reproducibility of the implemented system and to provide access to the complete firmware and supporting project files.